gt security guide — gt unlock and gt sandbox mode

Written against **gt v0.21.0** (gt-lotr 0.4.1); sections 1-9 describe 0.20.x and still hold, and section 10 adds what 0.21.0 brought. Unlock and sandbox mode both ship **off**. This guide says what each does when you turn it on, what it needs, how to set it up on macOS, Windows and Linux, and — just as plainly — what it does not protect. gt sandbox mode, which fences the assistant's own shell and file tools off the vault and gt's state, is section 8. MCP servers Claude Code connects to directly, outside both, are section 9.

"Unlock proves a person was present and limits what agents can do on their own. It is not anti-malware. If something already runs as you, it can wait for you to unlock."

1. The goal, and what was done to get there

gt aims to make everything it touches as secure as it can, and to tell you exactly how secure that is on your machine. An AI agent working for you runs as you: it can call tools, run shell commands and read your files. gt unlock puts a person back in the loop for the things that matter — your connected systems (LOTR), your credentials, publishing, and gt's own guards — without slowing down ordinary work.

Read this first. In 0.20.1 the authority runs **as you** (levels L1 and L2). Against a program running as you — and that includes the assistant's own shell — the gate is **friction**: it stops accidents and makes the assistant ask, but a determined process of yours can get around it (section 4 lists how). The guarantees that hold even then are narrower and come from hardware: at **L2**, opening a sealed credential needs your finger or PIN every time (or once per short window you choose), and a consent-tier LOTR operation can need a touch of its own. A real boundary against the agent needs **L3** — the authority as a separate, administrator-owned service — which **0.20.1 does not ship**.

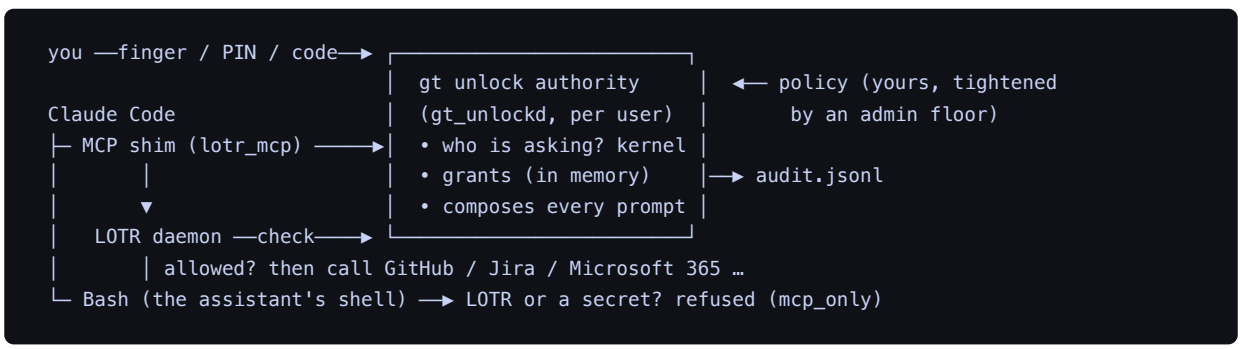
An independent security review of 0.20.1 (2026-10-03) reproduced seven bypasses of the gate by a same-user process. Each was fixed, and each is now a regression test; what cannot be fixed without L3 is written down in section 4 instead of being claimed away. A second, fresh-context review (2026-10-04) found more: the sandbox's inbox could be raced into copying a read-denied file (the TOTP seed) into the vault, an orphan could give itself a terminal and read without unlocking, an older signed policy approval could be replayed, a repository's own settings could loosen the sandbox unreported, the file tools could plant code where Claude Code or your shell loads it, and requests could name a session they do not belong to. Each is fixed or, where it cannot be prevented at L1/L2, reported and written down here; each demonstrated bypass is a regression test.

What was built, and what each piece really does:

MEASURE	WHAT IT DOES	AGAINST A PROCESS RUNNING AS YOU (L1/L2)
Kernel peer identity. Every caller is identified by the operating system (macOS audit token with the process version, Linux `SO_PEERCRED` + start time, Windows named-pipe client pid + creation time + user SID) — never by anything the caller says	A process cannot pretend to be another live process	Holds
The client checks the server. Before sending anything, gt's client asks the kernel which process serves the authority's address and refuses it unless it runs the installed `gt_unlockd.py`	A fake "allow everything" server bound after the real one stopped is refused	Friction: a process of yours can run the *real* `gt_unlockd.py` with its own state
Signatures, not booleans. Touch ID and Windows Hello sign a fresh challenge with a hardware key (Secure Enclave / TPM); gt verifies the signature itself in Python	A fake helper that answers "ok"; a replayed old approval	Holds for presence. Before each use the helper is read once, hashed against what install recorded, and run from a private copy of exactly those bytes, so a swap after the check runs nothing
Sealed credentials (`sealed:` refs). Encrypted at rest under a biometric hardware key; **every open needs your finger or PIN** (or once per `secrets_window_s`, per process and grant, never shared)	Credential theft from disk; reading a sealed value without you touching the sensor	**Holds at L2.** Once opened, the value is in the requesting program's memory
Biometric consent (`consent_requires_factor: platform`). A touch over each consent-tier operation, composed by the authority from the operation itself	A screen-control agent clicking "approve"	**Holds at L2**, as long as you read what the prompt names
The `mcp_only` door. LOTR and the credential broker serve only the MCP connection Claude Code started (the installed `lotr_mcp.py`, child of `claude`) — not a command run in the assistant's shell	The assistant using LOTR or your secrets through Bash by accident or by habit	Friction (section 4)
Only the authority asks. Every prompt is composed by gt's unlock authority, naming who is asking and for what; a caller's own reason appears only as a quoted, unverified claim. A client can request an unlock; it can never claim one	An agent typing "approved" on your behalf	Holds; a process can still show a look-alike prompt of its own
Grants in memory only, bound to one session and one seat (the session's vault MCP server holds its own grant, apart from LOTR's shim and the shell, so approving one does not unlock the other) and revoked on idle (15 min), TTL (8 h), screen lock, sleep, clock rollback, the session or MCP shim ending (a replacement shim starts with nothing), and `gt_unlock.py lock`	A forgotten unlocked session staying open	Holds for timing; see section 4 for who can use a live grant
Policy approval signed by your platform factor. A policy edited behind gt's back is not trusted until approved with a step-up, and on macOS /	Rewriting `policy.json` and	Friction: a process of yours can rewrite your enrolment too (section 4). TOTP-only (Linux): a hash, L1

MEASURE	WHAT IT DOES	AGAINST A PROCESS RUNNING AS YOU (L1/L2)
Windows the approval is a Touch ID / Hello signature over the policy itself, a monotonic serial and its time; the authority keeps the highest serial it has accepted, so an older signed approval restored with its policy fails closed	its approval to open everything	
Admin policy floor. An administrator-owned file that can only tighten; each scope's level is the stricter of yours and the admin's, each by its own most specific rule	A user pattern loosening an organisation's rule	Holds as logic; binding only at L3
Fail closed. With unlock on, an unreadable or missing policy, deleted policy files, a missing factor, an authority that is down or cannot be verified, an unwritable audit log (for writes and secrets), or a policy edited behind gt's back locks every gated action — never the reverse	"Broken" quietly meaning "open"	Friction: deleting the `unlock-on` markers too turns it off
No gt crypto store. gt brokers proven stores (1Password, sops+age, Vault, the OS keychains) and uses CryptoKit / DPAPI for sealing; it ships no cipher of its own	Home-made cryptography	Holds
gt sandbox mode (section 8, separate from unlock). Claude Code's own OS sandbox (Seatbelt / bubblewrap) around the assistant's shell, plus permission rules on its file tools, keep both off the vault, gt's state, the unlock home and LOTR, and the file tools off every file Claude Code, your shell or launchd loads code from; the vault is reached through gt's MCP server and write queue	The assistant's shell reading the TOTP seed, rewriting gt's policy, enrolment, hooks or plugin files, or writing the vault directly	**Holds on macOS, Linux and WSL2** for the shell and its children (an OS boundary) **while nothing loosens it**: a repository's own `.claude/settings*.json` can, unless the sandbox is admin-required through managed settings or `claude --settings` (section 8.4); `gt_sandbox.py check` reports every loosening key. Native Windows: permission rules only — friction
Every grant audited. Each unlock, check, secret read and revocation is one line in `~/.claude/golden-thread/unlock/audit.jsonl`, with its grant id — never a code or a secret value. A write, consent or secret release that cannot be recorded is refused	"What happened while I was unlocked?"	The log is yours: a process of yours can edit it afterwards

2. How it works



1. A session starts. gt registers it with the authority; the LOTR MCP connection Claude Code starts registers itself as that session's one shim. The authority accepts it only when the kernel says its parent is `claude` **and** the process runs the installed gt-lotr `lotr_mcp.py` (by real path, from Claude Code's `installed_plugins.json`) under an isolated interpreter (`python -I`, below). A second shim is refused while the first is alive; a shim that replaces a dead one starts with no grant. If the authority restarts, the shim registers again on its next call. On Windows the installer points the MCP command at the Python interpreter itself rather than at a launcher script (0.20.1): a script in between would be the parent, and every registration would be refused.
2. The assistant calls a LOTR tool. LOTR asks the authority whether **this process** (named by the kernel) may use `lotr:<connection>:<tier>`. Locked: the authority asks you — a Touch ID or Windows Hello sheet, then a dialog for your authenticator code — naming the requester and the scope.
3. You approve. The authority checks the signature and the code, grants the session, and LOTR proceeds. The grant id is written into LOTR's audit line for that call.
4. A sealed credential the call needs is opened by the authority with **another** touch (every time by default; `gt_unlock.py policy secrets-window 60` allows one touch per minute per process and grant). Only gt-lotr's installed `lotrd.py` may ask for a credential on behalf of the shim, and the value goes to the downstream call, never back to the assistant.
5. Fifteen idle minutes, eight hours, your screen locking, the lid closing, or the session ending: the grant is gone and the next call asks again.

Reads of LOTR connections can stay open (`read_without_unlock`, on by default) — for a person at a terminal. The requesting process must have a controlling terminal **and** a login shell above it (argv[0] starting with `-`, or `-l` / `--login` — what login(1), sshd, Terminal, iTerm2 and tmux start). An orphan (re-parented to launchd or init, no login shell above it) never gets it, even after giving itself a terminal with `setsid` + `openpty` + `TIOCSCTTY` (the 2026-10-04 re-review did exactly that against 0.20.1's first build; it is now a regression test). On Windows the parent chain must be intact and reach an interactive shell or terminal host. This is a convenience, not a boundary: a process of yours can start its own login shell (`bash -lc`, `script`, a scripted Terminal window). For a boundary set `read_without_unlock` false. Terminals that start non-login shells (GNOME Terminal's default) need an unlock for these reads. Writes and anything gated need the grant. Consent-tier operations still get LOTR's own confirmation. Consent is: the curated `send_mail` and `merge_pull`; the raw spellings of the same acts (`PUT /repos/o/r/pulls/N/merge` and `/repositories/ID/pulls/N/merge`, trailing slash or case aside; Graph `sendMail` and a message's `send`, `forward`, `reply`, `replyAll`, `createReply`, `createForward`; Graph `POST /$batch`, which can carry a send); a GraphQL mutation naming `mergePullRequest` or `enablePullRequestAutoMerge`; and an HTTP `DELETE` on any connection. Any other destructive endpoint on a generic REST connection is only a write unless you add `policy.consent` globs to the connection (for example `"consent": ["POST /v1/payments/*", "PUT /admin/*"]`, matched as `METHOD /path`). Pagination cursors are held in the daemon, bound to the connection, seat and operation, expire after ten minutes, and only `call_read` accepts one. and you can require a Touch ID / Hello approval per operation instead (`gt_unlock.py policy consent platform --window 300` approves consent operations for five minutes, then asks again). Stopping the authority while unlock is on needs a fresh factor; `gt_unlock.py lock` never does.

3. What it needs, and how to set it up

Everything below runs from your own terminal. `gt_unlock.py` lives at `~/.claude/golden-thread/hooks/gt_unlock.py` (written `gt_unlock.py` below).

K is how many factors must agree to unlock. The default is K = 2 with Touch ID (macOS) or Windows Hello (Windows) among them, and K = 1 (TOTP) on Linux. gt never lowers K on its own: when this machine cannot meet it, `policy enable` refuses and names the command that lets you choose — `gt_unlock.py policy enable --factors totp` sets K to the factors you name, records in the policy (and the audit log) that you chose it, and still needs your current factors to change. At a terminal, `policy enable` offers that choice itself. `gt_unlock.py status` shows K, the choice, and the one command to run next.

One code per command. When a request unlocks you and also needs a fresh confirmation (changing a security setting with `gt_settings.py set …`), the factors just collected for that same request are the confirmation — you are not asked twice. A code is still accepted once only: if a later command in the same 30 s window asks for TOTP again, it says "wait for the next code (about N s)". `gt_settings.py set unlock on|off` asks only `policy enable|disable` for your factors. On Linux / WSL2, `gt_settings.py set sandbox_mode on` checks for bubblewrap and socat before it asks for any code, and names the install command for your distribution.

MACOS — TOTP + TOUCH ID (L2)

Needs: a Mac with Touch ID, the Xcode Command Line Tools (`xcode-select --install` , so `install.sh` can build the Touch ID helper), and an authenticator app on your phone.

1. Re-run `install.sh` . It builds `~/.claude/golden-thread/bin/gt-presence` from the shipped Swift source and ad-hoc signs it (a Developer ID signed build is provided with releases).
2. `gt_unlock.py enroll touchid` — touch the sensor once. This creates two Secure Enclave keys that only work with your current fingerprints (adding or removing a fingerprint retires them, by design).
3. `gt_unlock.py enroll totp` — scan the QR code, type the code your app shows. The screen is cleared afterwards; the seed is shown only now.
4. `gt_unlock.py recovery` — print ten one-time recovery codes and store them offline.
5. `gt_unlock.py policy enable` — proves your factors work (touch + code), then turns unlock on.
6. `gt_unlock.py status` should say `level L2` . `gt_unlock.py verify` runs the self-check.

A Mac without Touch ID (a desktop Mac without a Magic Keyboard with Touch ID, or no Command Line Tools, so no helper): TOTP is what this Mac can use, and unlock runs at **L1**. `gt_unlock.py enroll totp` , `gt_unlock.py recovery` , then `gt_unlock.py policy enable --factors totp` (K = 1, recorded as your choice). If the Mac has Touch ID but you do not want to use it, enrol TOTP first with `gt_unlock.py enroll totp --without-platform` — accepted only from your own terminal (a process with a terminal and a login shell above it, and no Claude Code session), so an agent in a session cannot enrol factors it would then hold. It is the same kernel check `read_without_unlock` uses and, like it, friction rather than a boundary at L1/L2: a same-user process that detaches itself and starts its own login shell on a pseudo-terminal passes it. A TOTP enrolled that way shows in `gt_unlock.py status` , and your own `enroll touchid` would then ask for a code you do not have — so it cannot happen silently.

WINDOWS — TOTP + WINDOWS HELLO (L2)

Needs: Windows 10/11 with Windows Hello set up (Settings › Accounts › Sign-in options › PIN (Windows Hello); a fingerprint or face reader is optional), and an authenticator app.

1. `gt_unlock.py enroll hello` — Windows Security asks for your PIN or biometric a few times (key creation, a check that the key signs deterministically, which sealing relies on, and a final proof). The Hello dialog shows its own wording ("Making sure it's you"); gt shows the requester and scope in its own prompt.
2. `gt_unlock.py enroll totp` , `gt_unlock.py recovery` , `gt_unlock.py policy enable` as above.

Without Windows Hello (no PIN set up, or you prefer not to use it): Hello is optional. When Hello is not set up, `gt_unlock.py enroll totp` works straight away; when it is set up but you do not want it, use `gt_unlock.py enroll totp --without-platform` from your own terminal window (refused from Claude Code's shell, for the reason given under macOS). Then `gt_unlock.py recovery` and `gt_unlock.py policy enable --factors totp` . Unlock then runs at **L1**, and sealed credentials (which need Hello) are not available.

The Hello helper is a Windows PowerShell 5.1 script run with `-ExecutionPolicy Bypass` for its own process only; nothing about your machine's execution policy changes. It is not code-signed: the security rests on the TPM signature gt verifies, not on who signed the script. Organisations that block unsigned scripts sign it with their own certificate. Sealing on Windows needs a desktop logon (user-scope DPAPI is unavailable in an SSH key logon).

LINUX — TOTP (L1), OPTIONALLY ENTRA

Linux has no platform authenticator gt can use, so unlock there is **L1**: it stops agents and accidents, not malware running as you. `gt_unlock.py enroll totp` , `gt_unlock.py recovery` , `gt_unlock.py policy enable` (K = 1 is already Linux's default). Without a desktop, codes are typed in the terminal that asked. Enrolling and then making recovery codes inside one 30 s window asks you to wait for the next code; the message says how long.

GRANTS, AND KEEPING THE AUTHORITY CURRENT

`gt_unlock.py unlock` unlocks the shell (or Claude Code session) that asked — other terminals and sessions stay locked. Locking the screen, sleep, idling past the limit or `gt_unlock.py lock` ends it, and `gt_unlock.py status` says when and why ("revoked at 14:02: screen locked"). Re-running `install.sh` — an upgrade, a reinstall or a roll-forward — restarts the authority when its code changed (`gt_unlock.py daemon restart-if-stale`): that stop needs no factor because it only stops — it never grants — and every grant is dropped, so you unlock again. If it did not happen, `status` and `verify` say "the unlock service is

running old code (pid N, since ...)" with that command. Rolling back to a release older than 0.20.1 cannot do this for you: stop the authority first (`gt_unlock.py daemon stop`).

MICROSOFT ENTRA ID SIGN-IN (ANY PLATFORM, OPTIONAL)

A fresh browser sign-in (`prompt=login`) can be one of your factors: authorization code + PKCE on a `http://localhost` redirect, the ID token's signature and claims checked by gt (issuer, tenant, audience, nonce bound to the request, sign-in time). You register an app once:

1. Entra admin center › App registrations › New registration. Account types: your organisation only (strongest), any organisation (`organizations`), or work and personal accounts (`common`).
2. Redirect URI: platform *Public client/native (mobile & desktop)*, value `http://localhost` .
3. No client secret. API permissions: Microsoft Graph delegated `openid` , `profile` , `email` only. Leave *Allow public client flows* off.
4. Record the Application (client) ID and Directory (tenant) ID, then: `gt_unlock.py policy sso --client-id <app id> --tenant <tenant id|organizations|common>` and `gt_unlock.py enroll sso` .
5. Enforce MFA in Entra with Conditional Access (authentication strength) on the app; v2 ID tokens do not always say how you signed in. Sign in with Apple is not supported (it cannot redirect to a local address and carries no tenant or MFA claims).

4. What each level protects — and what it does not

`gt_unlock.py status` and `/gt:gt-doctor` (rows `unlock` and `security`) name the level this machine runs at. Nothing here says "secure"; each check passes, fails, or says it could not be checked.

LEVEL	WHAT IT IS	STOPS	DOES NOT STOP
off (default)	Nothing gated	—	Anything
L1 Gate	TOTP (and SSO), no platform factor	Accidents, use after the session ends or the screen locks, the assistant using LOTR or secrets *by the ordinary route*; full audit. **Friction, not a boundary**, against the assistant's shell	Any program running as you — the assistant's shell included — that sets out to get around it: it can read the TOTP seed and answer the code itself
L2 Sealed	Touch ID / Windows Hello required in every unlock; credentials sealed	Reading a sealed credential without your finger or PIN (each open, or each short window); a consent-tier operation without a touch, when you require one; screen-control agents clicking approve (they cannot touch the sensor). Everything else as L1 — **still friction** against a process running as you	A program running as you using what you approve: it can ask while you are there, race a look-alike prompt, use an unlocked grant, or get around the gate itself (below); code patched to capture your next unlock
L3 Separated	Authority and LOTR as a separate service account, code and policy administrator-owned	Use before unlock, policy downgrade, seed and credential reads, the gate bypasses below — **the agent boundary**	An attacker inside your session after you unlock. **gt 0.20.1 does not ship L3.**

THREAT	L1	L2
The assistant using LOTR or secrets from Bash *by the ordinary route*	Refused (`mcp_only`)	Refused
The assistant's shell, or any program of yours, setting out to get around the gate	**Not stopped** — friction only (list below)	**Not stopped** for the gate; sealed values still need your touch per open
Another local agent or script, while locked	Gated, but can read unsealed credentials directly	Sealed credentials stay sealed
Malware running as you, before you unlock	Not stopped	Partly: it cannot open sealed credentials without your touch, but can change gt's code for your next unlock
Malware running as you, after you unlock	Not stopped	Not stopped for the gate; each sealed open still needs a touch, which it can ask you for
A screen-control / accessibility agent	Can click dialogs	Cannot touch the sensor
A keylogger reading your TOTP code	First use wins (each code works once)	Same; use TOTP with a platform factor
A fake prompt racing gt's	Possible	Possible: gt's prompts are composed by the authority and name the requester and scope — read them
Stolen laptop, locked	Disk encryption + revoke on lock	The Secure Enclave / TPM needs your finger or PIN
Stolen laptop, unlocked and unattended	Until idle / TTL (15 min idle by default)	Same, but each sealed open needs a finger
Identity provider compromise	SSO forged; two factors including a local one mitigate	Same
Root / administrator compromise	Out of scope	Out of scope

How a program running as you can still get around the gate at L1/L2 — each is friction, documented, and closed only by L3. With **gt sandbox mode** on (section 8, macOS / Linux / WSL2), the assistant's *shell* is fenced off the files most of these routes need — the unlock home, gt's hooks and state, the plugin files, LOTR — but every other program of yours is not:

- **Run its own authority.** It can kill the daemon (a signal needs no factor) and start the *real* `gt_unlockd.py` against its own home at the same address; gt's clients check the program, not whose state it serves. A restart revokes every grant, so this shows up as a fresh prompt or a refusal, not silent access to yours.
- **Rewrite your files.** The policy, its approval, the enrolment, `installed_plugins.json` and gt's own code are your files. The policy approval is a Touch ID / Hello signature, but a process of yours can create its own Secure Enclave key, write it into the enrolment and sign with it (a Secure Enclave key is not bound to the program that made it). An approval by a key the enrolment marks as needing no finger is refused — the marking itself is your file. Deleting the policy and its state fails closed while the `unlock-on` markers exist; deleting the markers too turns unlock off. Restoring an *older* signed policy with its `state.json` does not work: the approval signs a serial and its time, and the authority refuses a serial below the highest it has accepted (kept in memory and in `approval-floor.json`). Approvals made before this check count as serial 0 until the first new one. Rewriting the floor file and restarting the authority rolls it back; while the authority runs, the in-memory floor holds.
- **Read the TOTP seed** (`totp.seed`). At L1 that is the whole unlock: it can compute a code and unlock its own session — and, being in the same session as the shim, the shim's.

- **Look like gt's own processes.** The shim seat and the credential consumer are checked by the real path of the file a process runs; a process of yours can run the real file with its own input. A replacement shim starts with no grant, and the first shim keeps its seat while it lives. The real file can also be made to run someone else's code: before 0.20.1's re-review fix, `PYTHONPATH` pointing at a `sitecustomize.py` ran arbitrary code inside the real file, under its trusted identity. Now every trusted Python process (`lotrd.py` , `lotr_mcp.py` , `gt_vault_mcp.py` , `gt_unlockd.py`) must also run under an isolated interpreter: gt starts each with `python -I` , and the authority and its clients refuse a peer whose command line lacks `-I` (or both `-E` and `-s`), or cannot be read. On macOS and Linux a refusal names the injection variables it saw, never their values; on Windows the environment is not read and only the flag check applies. **Start a hand-run lotrd with `python3 -I lotrd.py …`** — one started without it is no longer trusted while unlock is on. Not covered: a process of yours that modifies the interpreter or its site-packages (Homebrew's are writable by you).
- **Leave the session.** A double-forked process is not a descendant of `claude` , so the `mcp_only` door does not apply to it; it gets no read without unlock, and any unlock it asks for is a prompt naming it — even if it gives itself a terminal (above). On Windows, where there is no controlling terminal, its broken parent chain is what refuses it.
- **Claim a scheduled job's name** (`GT_JOB`) from outside a session and use that job's allow-listed, narrow, read-only scope. List only what you would accept that for.
- **Swap the Touch ID helper.** The helper is hashed at install. Each use reads it once through a no-follow descriptor, hashes exactly those bytes against the install record, writes them to a fresh private 0700 folder, checks the code-signing requirement on that copy, runs the copy and deletes it, so swapping the helper between the check and the run runs nothing. But the record is your own file: a process that rewrites the helper *and* its install record can watch what is sealed or opened afterwards (not forge your presence; a release binary's Developer ID signature is checked too), and a process of yours could in principle race the private copy.
- **Edit the audit log** after the fact, and the hooks' settings guard sees only Claude Code's Write/Edit tools, not a shell command writing `settings.json` .

Other honest limits:

- A brokered secret is only as locked as the weakest of the store, gt's grant, and the way it reaches the program. Environment variables are readable by any process of yours (`ps eww`). Only `sealed:` refs need your touch per open; `file:` , `keychain:` (L1) and the other stores are as strong as their own table row in section 5.4.
- The admin floor and the policy checks stop agents and accidents, not someone with your account; on a Mac where the daily account is an administrator, even L3's admin-owned files are bounded by the administrator password prompt.

5. Locking down more with gt unlock

5.1 THE VAULT: LOCK CHOSEN FOLDERS, NOT THE VAULT

`gt_lock.py add Secrets/aws.md --vault <vault>` encrypts a file with age to `aws.md.age` and removes the plaintext; a `.gt-locked` note in the folder says what is locked and how to open it. `gt_lock.py open Secrets/aws.md.age --vault <vault> | less` decrypts to a pipe only (a terminal needs `--show` ; a file is always refused) after a `gt:lock:<folder>` grant; `gt_lock.py restore` puts the plaintext back (mode 600); `gt_lock.py status` lists what is locked and how strong each lock is. The identity comes from `--identity` , `$GT_AGE_IDENTITY` , `SOPS_AGE_KEY_FILE` , or `~/.config/sops/age/keys.txt` . Lock before the first commit or sync: plaintext already in git history, sync version history or backups stays there.

- **Strength:** L2 only when every recipient is hardware-held (`age1se1…` Secure Enclave via `age-plugin-se` , `age1yubikey1…`); **L1 when any recipient or the identity is a plaintext key** such as `~/.config/sops/age/keys.txt` — the usual setup. Never L3: once opened, any process of yours can read the plaintext.
- `core-rules/` and `global-memory/` can never be locked (enforced in code): gt reads them on every turn, and locking them would silently switch Core-rule injection off.
- Why not the whole vault: Obsidian keeps it open, gt's hooks read it on every prompt, and an encrypted volume synced by Dropbox and mounted on two machines can corrupt.
- What others see: Obsidian shows links into a locked file as unresolved; Dropbox, iCloud and git see only `.age` ciphertext (commit only ciphertext); gt's lint, tasks and recall treat a locked folder as absent and say so once.
- An encrypted disk image (macOS sparse bundle) or BitLocker VHDX is a bring-your-own option; once mounted, any process of yours can read it.

5.2 OTHER APPS' CREDENTIALS, THROUGH THE BROKER

Credentials are released only under a grant, and handed over by pipe or a short-lived 600 file, never by environment variable unless you ask for it.

- **git / GitHub publishing:** store the token sealed (`gt_unlock.py seal put github` , value on stdin), map the host in `~/.claude/golden-thread/unlock/credentials.json` (`{"git": [{"protocol": "https", "host": "github.com", "username": "x-access-token", "ref": "sealed:github"}]}`), and set `git config --global credential.https://github.com.helper "!python3 ~/.claude/golden-thread/hooks/gt_unlock.py git-credential"` . Every push then needs a `gt:publish` grant. It is real only if the token exists nowhere else (no plaintext `~/.config/gh/hosts.yml` , no unencrypted SSH key with the same reach).
- **AWS:** `credential_process = python3 ~/.claude/golden-thread/hooks/gt_unlock.py aws-credential --ref sealed:aws` (the sealed value is the JSON AWS expects).
- **Any command:** `gt_unlock.py run --scope gt:publish --secret-file GH_TOKEN_FILE=sealed:github -- <cmd>` hands the command a path to a 600 file deleted when it exits. `--env VAR=REF` puts the value in the environment instead, which is weaker, and gt says so each time.
- kubectl exec plugins and docker credential helpers can call `gt_unlock.py secret get REF` the same way (it refuses to print to a terminal).
- Moving an existing credential in: `gt_unlock.py seal migrate file:/abs/path --name github --remove-source` (also `store:` and `keychain:`).

5.3 SSH KEYS

Keep SSH keys in hardware and use them per signature: Secure Enclave keys through an agent such as Secretive (macOS) or a FIDO2 security key (`ssh-keygen -t ed25519-sk`). gt does not hold SSH keys; an unencrypted key file in `~/.ssh` is readable by any process of yours whatever gt does.

5.4 BRING YOUR OWN SECRET STORE

Refs `sops:` , `op:` , `bw:` , `vault:` and `wincred:` are resolved by the authority under a grant, and `gt_unlock.py status` / doctor label each with its real level:

STORE	LEVEL AGAINST PROCESSES RUNNING AS YOU
Keychain via `/usr/bin/security` (LOTR's `keychain:` refs)	**L1**
Windows Credential Manager / DPAPI (`wincred:`)	**L1**
sops + age, identity in a plaintext file	**L1**
sops + age, identity in the Secure Enclave or a YubiKey	**L2**
1Password CLI (`op:`)	**L2** (its own biometric unlock and idle limit)
Bitwarden CLI (`bw:`)	**L1** after unlock (the session key sits in the environment or a file)
HashiCorp Vault (`vault:`)	**L1** (0.20.1 does not hold Vault tokens in the authority)
gt's sealed store (`sealed:`)	**L2**

5.5 SCHEDULED JOBS

Nothing is allowed unattended by default. Each job may be given one narrow scope: `gt_unlock.py policy unattended add <job> lotr:<connection>:read` (or `secret:<ref>` for one credential), and the job's definition sets `GT_JOB=<job>` . Write, consent, publish, wildcard and gt's own scopes can never be allowed unattended; jobs are never prompted.

5.5A PUSHES AND COMMITS: A FINGERPRINT, AND WHAT IT PROVES (0.20.4)

Three settings, all off by default, for the repos named in `push_fingerprint_repos` :

SETTING	WHAT IT DOES	WHAT ENFORCES IT
`push_fingerprint`	a pre-push hook asks for `gt:publish` (one fresh Touch ID on macOS, or Windows Hello on Windows, per push)	the hook only: `git push --no-verify` skips it
`push_fingerprint_seal_token`	the HTTPS push token for each GitHub host moves into the sealed store; gt's credential helper in the guarded repo releases it to git under `gt:publish`	gt unlock, for git's normal path: `--no-verify` still needs the touch, and a process from Claude's shell is refused the token before any prompt. **Not** against a process running as you that sets its own helper (`git -c credential.<url>.helper=… push`) while gh is still signed in (see below)
`commit_fingerprint` (macOS)	a Secure Enclave key signs every commit and tag; each signature needs Touch ID	the server, once `gt_sign.py github --apply` has added a ruleset requiring signed commits on the default branch with **no bypass actors** (an admin's unsigned push is refused too)

`commit_fingerprint` is macOS only in 0.20.4: Windows Hello commit signing is planned for 0.20.5. Hosts come from each repo's own remotes: github.com or a GitHub Enterprise server. An SSH remote has no token to seal, so the hook is its only push guard.

What it does not prove or stop: * A touch proves you approved, not that you read the change. * gh keeps its own token in the keychain, and setup needs gh signed in. While it is, a process running as you can push **from any repo, the guarded ones included**, by pointing git back at gh's own helper for one command (`git -c credential.https://github.com.helper= -c credential.https://github.com.helper='!gh auth git-credential' push --no-verify`): no touch, no gt involved. The sealed token closes git's normal path, not that one. `gh auth logout -h <host>` after setup closes it, at the cost of using gh there; that is your choice, never automatic. * The same gh token, with admin rights on the repo, can delete the ruleset. Server enforcement holds against a push, not against someone who holds an admin token for the repo. * The sealed token can also be read from your own terminal with `gt_unlock.py secret` and one touch; `gt:publish` is what gt's helper asks for, not the only way to the sealed store. * The ruleset covers the default branch only. A local commit can still be made unsigned (`-c commit.gpgsign=false`); it cannot reach the default branch. * Commits GitHub itself signs (web edits, merges through the UI) count as signed. * With unlock already on, `push_fingerprint on` keeps your policy as it is and makes push, tag and the unlock policy at least step-up (a `deny` stays `deny`). With unlock off, it turns unlock on with the scopes gt lists open; anything unlisted keeps the policy's default level.

`off` undoes each setting exactly: a credential helper or hook of your own is put back as it was. `off` asks for a fresh confirmation first (when unlock is on), then deletes the sealed copies and restores the policy, and only then removes the hooks and helpers: if any step is refused (for example from Claude's shell), it says so, **leaves the guard in place**, and exits non-zero; run it again from your own terminal. The hook is written where git looks for hooks, so with a shared `core.hooksPath` (a global hooks folder, or a tracked `.githooks/`) it applies to every repo that uses that folder. The signing key stays registered on the server, so past commits stay Verified; remove it there yourself if you want to.

5.5B SETTING IT UP, PROVING IT, AND BACKING IT OUT

Run these in **your own terminal**, not inside Claude Code: enrolling a factor and changing the unlock policy are meant to need you. The repo is any git repository you push from.

```
H=~/.claude/golden-thread/hooks
REPO=/absolute/path/to/your/repo
```

Before you start 1. `git -C "$REPO" config user.email` prints the address your Git host knows you by. A signature is matched to it. 2. `ls "$REPO/.git/hooks/pre-push"` finds nothing. If you have your own pre-push hook, `on` refuses and changes nothing. 3. `python3 $H/gt_unlock.py status` : if unlock is **off**, `on` turns it on with every other scope open. If it is **already on**, your policy is kept and only push, tag and the policy itself become step-up.

Set it up 1. A factor, once: `python3 $H/gt_unlock.py enroll touchid` (Windows: `enroll hello`). 2. The repo: `python3 $H/gt_settings.py set push_fingerprint_repos "$REPO"` (comma-separate several). 3. Optional, the lock behind the hook: `python3 $H/gt_settings.py set push_fingerprint_seal_token on`. It needs `gh` signed in to the repo's host (`gh auth login -h <host>`), and pushes the assistant starts are then refused. 4. `python3 $H/gt_settings.py set push_fingerprint on`. 5. macOS: `python3 $H/gt_settings.py set commit_fingerprint on`. It prints a public key line (`ecdsa-sha2-nistp256 …`). 6. The server half: `python3 $H/gt_sign.py github` shows the plan on your repo's host, and `--apply` asks before it registers the key as a **signing key** and adds the no-bypass ruleset. It needs `gh auth refresh -h <host> -s admin:ssh_signing_key` and admin rights on the repo. Without `gh`, register the key by hand (Settings → SSH and GPG keys → New SSH key → Key type: Signing Key) and add a branch rule requiring signed commits.

Prove it * `python3 $H/gt_push_guard.py check` (every line PASS) and `python3 $H/gt_sign.py check --live` (one touch, PASS). * On a scratch branch: `git commit --allow-empty -m test` asks for Touch ID, and `git verify-commit HEAD` reports a good signature. A second commit where you press **Cancel** must fail with `gt_sign: not signed (cancelled)`. * `git push origin <scratch-branch>` asks before it goes, and the host shows the commit Verified. Delete the branch afterwards (that asks too).

Back it out, switches first: 1. `python3 $H/gt_settings.py set commit_fingerprint off`: the earlier git config comes back; the key is kept. 2. `python3 $H/gt_sign.py github --remove`: the ruleset goes; the key stays registered, so past commits stay Verified (remove it on the host yourself if you want). 3. `python3 $H/gt_settings.py set push_fingerprint off`: the hook, credential helpers and sealed token go, and the unlock policy is restored exactly as it was. 4. Check: no `pre-push` in the repo, `git -C "$REPO" config --local --get-regexp '^(gpg|commit\.gpgsign|tag\.gpgsign|user\.signingkey|credential)'` back to what it was, and `python3 $H/gt_push_guard.py check` saying it is off (exit 2). 5. Optional: `python3 $H/gt_unlock.py unenroll touchid`.

5.6 WHAT GT UNLOCK CANNOT LOCK

- Applications that never ask gt: a browser, an editor, any program reading its own config.
- Anything after you unlock, inside the unlocked window, from software already running as you.
- Credentials that also exist somewhere else in plaintext.
- Files outside the folders you lock.
- MCP servers Claude Code connects to directly, not through LOTR (section 9).
- What the gt-lotr domain agents can and cannot do (below): a split that stops a poisoned page steering an agent, not a boundary against your own session.

gt-lotr agents: a reader/writer split, and what it does not enforce (0.20.2)
gt-lotr ships eight plugin agents split so that no one agent holds private data, untrusted text and a way to act. The four **readers** (`jira`, `m365`, `github`, `runner`) are given `find` and `call_read` only, so a mail, an issue or a page that says "now send this" or "merge it" reaches an agent with **no write or consent tool to call**. That is a limit on what the agent can *call*, not on what an op does: the gateway rates a built-in connection's ops from its own profile, but for an `mcp` connection it trusts the downstream's tool names and `readOnlyHint`, so a hostile MCP server can mislabel a change as a read and a reader would call it. The readers are told never to call an op whose name says it changes something, whatever `find` rates it; that is an instruction, not a check. The four **writers** (`jira-writer`, `m365-writer`, `github-writer`, `writer`) are given `find` and `call_write` (and `call_consent` except `jira-writer`) and **no `call_read`**, and are **told not to read**. The gateway enforces that with **strict tool tiers** (on by default): each tool carries only its own tier, so a read-tier op sent through `call_write` or `call_consent` is refused with `wrong_tool` before anything is sent, as is a write through `call_consent`. The setting is per connection in `registry.json` (`"strict_tools": false` restores the older rule, where `call_write` also carried reads and `call_consent` carried everything; it is an admin-plane file, like the rest of a connection's policy), and on a connection where you turn it off "a writer reads nothing" is its instruction again. Jira's JQL search by `POST` and Graph's `POST /search/query` are rated read, so readers can use them and writers cannot. Each writer performs the one operation in an exact `SPEC` that the main session composed from *your* words, treats every payload word as data, asks consent at most once and never retries a refusal. "One operation" is likewise the writer's instruction: `maxTurns: 4` does not cap parallel tool calls (the writers are told never to issue any), and nothing at the gateway checks that a writer made only one.

What consent covers. gt unlock's platform consent (Touch ID / Windows Hello over the operation) applies to what the gateway rates as the **consent tier**: the curated ops `send_mail` (Graph) and `merge_pull` (GitHub) at their path templates; an MCP tool the gateway rates consent by its name or hint; and, since gt-lotr 0.3.0 (gt 0.20.1), the raw routes that do the same thing by another name: `POST /me/sendMail` or `/users/X/sendMail`, a message's `send`, `forward`, `reply`, `replyAll`, `createReply`, `createForward`, Graph `POST /$batch`, a merge through a path with a trailing slash or `/repositories/{id}/…`, a GraphQL mutation naming `mergePullRequest` or `enablePullRequestAutoMerge`, and an HTTP `DELETE` on any connection. A writer's

`call_write` cannot perform any of them. What it does **not** cover: any other destructive endpoint on a generic REST connection, which is write tier unless you add `policy.consent` globs to that connection.

This is **not a security boundary**. The agents share the session's own gt-lotr connection, grants and permission rules, and the main session can call the same tools: a compromised or fooled main session can still write, and the read-then-act path through it is guarded by rules in its instructions and your attention, not by the platform. When a write's target came from a read (an issue key, a transition id), the main session is told to show you the resolved target (connection, key, summary) before the write; a merge's head sha is never taken from a read. Strict per-connection isolation is the 0.21 proxy design.

WORKED EXAMPLE (MACOS)

```
gt_unlock.py enroll touchid && gt_unlock.py enroll totp && gt_unlock.py recovery
gt_unlock.py policy enable
printf %s "$GITHUB_TOKEN" | gt_unlock.py seal put github     # then delete the plaintext copy
lotr --home ~/.config/gt-lotr/personal add-http github@personal --profile github \
  --base-url https://api.github.com --identity me --auth bearer --token-ref sealed:github
gt_lock.py add Secrets/aws.md --vault ~/Vault
gt_unlock.py verify
```

WORKED EXAMPLE (WINDOWS, POWERSHELL)

```
$u = "$env:USERPROFILE\.claude\golden-thread\hooks\gt_unlock.py"
python $u enroll hello; python $u enroll totp; python $u recovery
python $u policy enable
python $u seal put github          # type the token at the hidden prompt
python $u verify
```

6. Enterprise

- **Admin policy floor:** `/Library/Application Support/gt/unlock-policy.json` (macOS, root-owned with root-owned parents, or delivered by an MDM profile that writes it), `%ProgramData%\gt\unlock-policy.json` (Windows, owned by Administrators/SYSTEM with no user write access) or `HKLM\SOFTWARE\Policies\gt\UnlockPolicy` (a JSON string, for GPO/Intune), `/etc/gt/unlock-policy.json` (Linux). Its existence turns unlock on. It only tightens: more factors, fewer allowed factors, shorter TTL and idle, stricter scopes, `locked_keys` the user cannot change, a pinned Entra tenant. A floor file gt cannot verify locks everything.
- The floor is **binding only at L3**, when the enforcing code is also administrator-owned; 0.20.1 runs the authority as the user. Say so in your own policy documents. Where users are administrators of their own machines, admin-owned protection is bounded by that password.
- **Signing:** release builds of the macOS helper are Developer ID signed and notarized. The Windows Hello script is unsigned by design; sign it with your own certificate where policy requires signed scripts.
- **Recovery for managed fleets:** keep `admin_only` recovery in your process: reset an enrolment by removing `~/.claude/golden-thread/unlock/` and the marker beside it, `~/.claude/golden-thread/.unlock-unlock-on` (which also discards sealed credentials), under your own controls.

7. Recovery, and turning it off

- **Lost phone or changed fingerprints:** `gt_unlock.py unlock --recovery` accepts one recovery code as one factor; you then re-enrol (`gt_unlock.py enroll …`). Each code works once.
- **Lost every factor:** remove `~/.claude/golden-thread/unlock/` **and** the marker beside it, `~/.claude/golden-thread/.unlock-unlock-on` (while either remains, gt treats unlock as on with no policy and keeps everything gated locked), then enrol again. Sealed credentials are lost with it — that is what sealed means; keep their originals in your store.
- **Turn it off:** `gt_unlock.py policy disable` (needs your factors) or `gt_settings.py set unlock off`. Off, every hook behaves exactly as before 0.20.1.
- **Check it:** `gt_unlock.py status`, `gt_unlock.py verify`, `gt_unlock.py audit -n 50`.

7a. Lockdown levels: how much Claude may run without asking (0.20.5)

How much Claude may run without asking. Most of the friction in a session is approval prompts; a lockdown level is a set of Claude Code allow rules that gt writes into `~/.claude/settings.json` for the level **you** choose, so routine work stops asking. The installer asks for it (`./install.sh --lockdown <level>` to answer in advance; with no terminal a new install stays strict and an upgrade keeps its level). Change it any time with `/gt:gt-settings lockdown <level>` (`gt_settings.py set lockdown <level>`), then restart Claude Code. gt writes and removes only its own rules, records them, backs `settings.json` up first and logs every change (`~/.claude/golden-thread/lockdown/`); uninstall takes them out. With gt unlock on, changing the level needs a fresh confirmation. gt never changes the level by itself.

LEVEL	RUNS WITHOUT A PROMPT	STILL ASKS
strict	nothing added -- exactly what Claude Code asks today (default)	everything Claude Code asks today
trust local	read-only shell (ls, cat, grep, wc, ...) and git (status, log, diff, show, branch); edits in the project; running tests; git add/commit	network, remote hosts, pushes, installs, deletes, anything not listed
trust network	trust local, plus ssh/scp/rsync, git push/pull/fetch, curl/wget, npm/pip/brew install, web fetch	any other shell command or edit outside the project
insecure	trust network, plus any shell command, any edit, web search	LOTR send/merge/delete still ask for consent

At every level above strict, Claude's Read tool is denied credential files (ssh keys, cloud and gh credentials, Claude Code's login, .env). That guards the Read tool only: from trust local up a shell command can still print a file, so Core rule 3 (no secret's value in the session) remains the line. These levels reduce prompts; they are not a security boundary.

Paranoid, the top rung (0.21.0). The ladder offers a fifth choice, `paranoid` : strict, plus the `paranoid` switch that locks gt's own installed code (section 10.1). It is not a stored level. Choosing it applies strict and turns the switch on, and it shows as paranoid whenever both hold. Leaving it for another level turns the switch off, with the step-up that already needs: gt unlock, or a person at a terminal. Turning it on needs gt unlock set up first, unless `paranoid.require_unlock` is off. The installer no longer asks the separate paranoid question. `--paranoid on|off` still sets the switch on its own, for example trust-local plus paranoid; `--lockdown paranoid --paranoid off` is refused.

8. gt sandbox mode

`gt_settings.py set sandbox_mode on` (off by default). gt then configures **Claude Code's own sandbox and permission rules** so that the assistant's shell and file tools cannot touch the vault or gt's state, and the vault is reached only through gt's channels: its MCP server and the write queue. It is independent of unlock — on together, unlock also gates the MCP server's reads (`gt:vault:read`) and writes (`gt:vault:write` , never opened by `read_without_unlock`), both under door `mcp_only` .

gt's MCP tools never edit `design.md` or anything under `global-memory/` (owner decision, 2026-10-04) — with sandbox mode on or off. Every gt MCP tool that can write refuses them with one line — "design.md and global-memory/ are changed by you, in an editor" — and checks **before** it asks the unlock authority for anything, so a refused target does not cost you a fingerprint. The check is the write broker's own shared helper (`gt_review_target.py` , 0.20.1), by **identity** and by **name**, and it also refuses a symlink that dangles or leaves the folder it sits in. Identity: any existing part of the path that the file system says is the same folder as `global-memory/` , or a final component that is the same file as the `design.md` beside it (a hard link, a bind mount, a case-insensitive or Unicode-normalising volume's other spelling, a Windows 8.3 name, a symlink); a project folder must also resolve to itself, so a project that is a link is refused. Name: the path as any file system may read it — NFKC and case-folded (so the long s `defign.md` and fullwidth letters match), with invisible characters, leading and trailing whitespace, trailing dots and spaces and an NTFS `::$DATA` suffix removed. A scaffold run for a new project never follows a symlink (it creates files exclusively). The two terminal-only tools that can touch them — `vault_init.py` (the new-project scaffold) and `gt_optimize.py --demote` — are yours to run. The broker still escalates a write it is handed (a request a shell leaves in the sandbox inbox): that becomes a `#conflict` task with a conflict file holding every version, and **you** apply it, in an editor or a terminal. (The broker's own check is by exact name; moving it to the same identity check is tracked separately — until then a case-insensitive volume's alias of `design.md` , sent by a shell, is not escalated by the broker.) Nothing in the sandbox-mode design depends on an automated way to apply one. The exceptions are deliberate: a read-only report may name them, `vault_events_emit` records paths as names,

and `vault_project create` makes a new project's `design.md` from the scaffold (a new file, by the existing create-project script — never an edit).

Every skill works under it (0.20.2). Each vault step a skill used to run from the shell has a route: a gt-vault MCP tool that runs the same script with the same checks (8.2), or — for the steps that change settings, install state or the whole vault — an honest "run this from a terminal". Each SKILL.md carries the table, and a test holds every skill of every gt plugin to it. Sandbox mode is no longer a preview.

Preview in 0.20.1. The fence is what this section says it is. What is not finished is the work inside it: most skills still run gt's vault scripts from the assistant's shell, and the sandbox refuses those. **Works under sandbox mode:** `/gt:gt-open`, `/gt:gt-query` and the writes of `/gt:gt-work` (through the gt-vault MCP), every `vault_*` MCP tool, and `gt_write_queue.py` (it leaves its request in the inbox). **Does not work yet:** the shell steps of the other skills — `gt_log.py add`, `gt_tasks.py`, `gt_task.py`, `gt_adr.py allocate`, `gt_closeout.py`, `gt_lint.py`, `gt_promote_detect.py`, `vault_init.py`, `gt_broker.py drain`. Each refused tool prints **one line**: what was refused, why ("sandbox mode: the vault is written only through the queue / gt-vault MCP"), and the next step — the MCP tool where one exists (`vault_queue_drain` for the drain), otherwise the exact command, with real paths, to run from a terminal. Nothing is left half-written (no sidecar or pending file to replay). With `sandbox_vault_reads deny` the vault copies of the tools cannot even be opened, so Python itself refuses them; each skill says what to use instead. Full skill support is planned for 0.20.2.

Before 0.20.1 these tools died in Python tracebacks, and `gt_log.py` advised Full Disk Access — advice for a different refusal (macOS refusing an interpreter a file). The two are told apart by where the tool runs: sandbox mode on and the process is Claude Code's.

8.1 WHAT IT WRITES

Into `~/.claude/settings.json`, through `gt_sandbox.py`, which records every entry it adds (in `~/.claude/golden-thread/sandbox/state.json`) so that turning it off removes exactly those and nothing of yours. Every key is one the Claude Code documentation names (code.claude.com/docs/en/sandboxing, /settings-reference, /permissions, read 2026-10-03):

KEY	VALUE	WHY
`sandbox.enabled`	`true`	The OS sandbox around t Bash, PowerShell and Mo tools and every process t start
`sandbox.allowUnsandboxedCommands`	`false`	No "retry outside the sandbox" escape hatch: `dangerouslyDisableSan` is ignored
`sandbox.failIfUnavailable`	`true`	Claude Code refuses to s rather than silently run commands unsandboxed
`sandbox.filesystem.denyWrite`	the vault; `~/.claude/golden-thread` (hooks, state, the unlock home and its `unlock-on` markers); the unlock socket's directory when it is outside that; `~/.config/gt-lotr` and LOTR's store (`~/.secrets`); `~/.claude/plugins` ; `~/.claude/settings.json` ; `~/.claude/vault-config.json`	The shell cannot change vault, gt's hooks and poli the plugin files the autho trusts, or the settings tha this off
`sandbox.filesystem.denyRead`	the unlock home (sealed store, `totp.seed` , recovery codes, audit log); LOTR's home and store; every **locked** vault folder; Claude Code's own login file `~/.claude/.credentials.json` (0.20.1; its OAuth tokens on Linux, WSL and Windows — macOS keeps them in the Keychain, and the entry is written anyway); and the **whole vault** while `sandbox_vault_reads` is `deny` (the default)	The shell cannot read the second factor, credentials Claude Code's login, or th vault
`sandbox.filesystem.allowWrite`	`~/.gt-inbox` ; `~/.gt-scratch`	The two writable places. `gt_write_queue.py` leav its requests in `~/.gt-inbox/queue/` . `~/.gt-scratch/<run>/<stage>-<unit>/` is the private sc folder of a pipeline stage agent whose tools can wr (0700 at every level; `gt_scratch.py`). Which agents qualify is derived each stage spec's tool lis against one list, `gt_agent_spec.WRITE_T` today only verify (a shell) does, and a read-only sta gets no folder and no scr text in its prompt. Interme files go there instead of t vault spool, nothing reads them back as instructions a finished run's folder is removed. Neither is inside vault, so `denyWrite` still covers all of it

KEY	VALUE	WHY
`permissions.deny`	`Edit(//<path>/**)` for every write-denied path, `Read(//<path>/**)` for every read-denied folder and `Read(//<path>)` for a read-denied file, and `Edit` on any `.claude/settings.json` / `.claude/settings.local.json`; plus `Edit` on every file Claude Code, your login shell or launchd loads code or configuration from: `~/.claude.json`, `~/.claude/{settings.local.json, CLAUDE.md, agents, skills, commands, hooks, output-styles}`, any `.mcp.json` and any project `.claude/{agents,skills,commands,hooks,workflows}`, the shell rc files (`.zshrc`, `.zshenv`, `.zprofile`, `.zlogin`, `.bashrc`, `.bash_profile`, `.bash_login`, `.profile`), `~/Library/LaunchAgents` (macOS) and `~/.config/systemd/user` / `~/.config/autostart` (Linux, WSL2)	Claude's file tools (Read, Write, NotebookEdit, Gre Glob) run **outside** the sar and follow permission rul instead; without the seco group they could plant cc that runs later outside ev fence. Not all of `~/.clau` denied: Claude Code's ov auto-memory and plans li there
`permissions.disableBypassPermissionsMode`	`"disable"`	No `--dangerously-skip-permissions` / bypass m while the fence is up; rest to its earlier value when t mode is turned off

`Write(...)` and `NotebookEdit(...)` path rules are not written: Claude Code checks file permissions against `Edit(path)` and `Read(path)` rules only, and an `Edit` deny covers every editing tool. Managed settings are never written; if they override what gt asked for, `gt_sandbox.py check`, `gt_unlock.py verify` and `/gt:gt-doctor` say so.

Settings merge, so other files can loosen the fence. A repository's `.claude/settings.json` (loaded from the working directory) and `.claude/settings.local.json` (loaded from the git root) merge with your user settings, and can loosen the sandbox in that project: `sandbox.enabled false`, `allowUnsandboxedCommands true`, `excludedCommands`, `ignoreViolations`, Unix-socket and Mach-lookup entries, `enableWeakerNestedSandbox` / `enableWeakerNetworkIsolation`, `additionalDirectories`, or an `allowRead` / `allowWrite` / Edit or Read allow rule that re-opens a region gt denies (a narrower allow wins over a wider deny). `gt_sandbox.py check` reports every such key as a problem, naming the file — in your user settings too, so a user `excludedCommands` is no longer a pass. It **reports; it cannot prevent.** Claude Code ignores a repository's loosening keys only when the sandbox is **admin-required**: `allowUnsandboxedCommands: false` set in **managed settings** or passed with `claude --settings <file>` (Claude Code v2.1.285 or later; user settings alone never make it admin-required). `gt_sandbox.py managed` prints that configuration for `managed-settings.json`, or saves it with `--out <file>` for `claude --settings`; gt never writes managed settings itself.

8.2 HOW THE VAULT IS REACHED

- **Reading:** gt's MCP server `gt-vault` (shipped in gt's plugin manifest; MCP servers run outside the sandbox) offers `vault_search` (index.md first, then pages scored like `/gt:gt-query`, current before expired before superseded), `vault_read` (size-capped at 64 KiB by default, 256 KiB at most, pageable) and `vault_list`. It is read-only, never returns a file outside the vault or inside a locked folder — even named exactly, and for `vault_search` even when a symlink in the vault points there (each result, and `index.md`, is resolved and checked the way `vault_read` checks a path) — and marks every result as untrusted data. With unlock on, each read needs `gt:vault:read`, served only to the session's registered `gt_vault_mcp.py` (checked from the kernel and by the installed file's real path, like LOTR's shim); a process from the assistant's shell is refused. By default `read_without_unlock` opens it like a LOTR read; set it false to need an unlock.
- **Writing:** `vault_queue_write` takes `gt_write_queue.py`'s ops and validation and queues the write; the broker decides it at once (apply, deduplicate, held, escalate, reject) and the result says which. From the shell, `gt_write_queue.py` cannot write the vault's queue folder, so it leaves the request in `~/.gt-inbox/queue/` — even when the sandbox refuses a `stat` of the vault itself (0.20.1); `vault_queue_drain` (or `gt_broker.py drain` in a terminal) moves it into the queue. **No hook drains the queue**, at SessionStart, Stop or anywhere else: under sandbox mode the drain runs outside the sandbox only through the gt-vault MCP or from your terminal, and the session-start `WRITE QUEUE` line says so instead of naming the shell command (which the sandbox refuses). The inbox is writable by anything in the sandbox, so the broker treats each file as untrusted data.

It opens the inbox folder once, refusing it if it (or `~/.gt-inbox`) is a symlink, and opens each file **relative to that folder handle with no symlink following** (`O_NOFOLLOW`; a FIFO cannot block it), checks the **open file** — a regular file with exactly one link, within the size cap — and reads it once, bounded. Those bytes are the only ones it uses, for accepting and rejecting alike, and it removes the file through the same folder handle; it never re-opens an inbox path. So a sandboxed writer racing the broker cannot swap in a symlink, a hard link or another folder to have a file it may not read copied into the vault (the 2026-10-04 review demonstrated exactly that against 0.20.1's first build, with the TOTP seed; it is now a regression test, deterministic and live under the sandbox runtime). A request must name this vault and pass the queue's validation (paths inside the vault, no generated files, Sources or Core rules, a plain request id). The broker then **stamps** it: origin `inbox` (a body that said `farm` keeps `farm`, which is stricter) and session `unknown-inbox`, whatever the body claimed — a writer in the sandbox cannot speak for a session, so **no live session's claim lets an inbox request through** (Core rule 1), not even the claim of the session running the drain; it waits until the claim ends. It is then decided like any request — conflicts and `design.md` / `global-memory` go to you. Those two are recognised **by identity, not by spelling** (0.20.1): a file named `design.md` in any case, Unicode form, trailing-dot/space, `::$DATA` or 8.3 (`DESIGN~1.MD`) spelling, anything under the vault-root `global-memory/` by those spellings, and anything that reaches either through a symlink or the same inode. A symlink in the path that dangles or leaves its own directory is escalated as well. Look-alikes (`xdesign.md`, `global-memory-x/`) are ordinary files. Rejected inbox files are kept in `spool/broker/rejected/` up to 50; past that they are logged and deleted. An oversized file is never read and is left for you. `vault_queue_write` stamps its requests the same way, with the MCP server's own session id: a tool call cannot name another session to write through its claim. A claim is matched **by identity, not spelling** (0.20.1): a case, Unicode, trailing-dot, 8.3 or symlink variant of a claimed path is held like the claimed path itself — on every platform, so on a case-sensitive volume a claim on `research.md` also holds `Research.md`, deliberately. Native Windows has no sandbox, so there the inbox is not a boundary at all: the broker still refuses links and reparse points and checks that the open file is the one it listed, inside the inbox, but Claude's shell can open your files directly anyway.

- **A skill's vault scripts (0.20.2):** the operation tools run them for the assistant — `vault_report` (read-only reports: tasks, handoffs, lint, close-out signals, promotion candidates, memory contradictions, digests, supersession, ADR lineage, entities, link suggestions, optimize, upgrade status, broker status, doctor, context, wiki lint), `vault_task`, `vault_tasks_regen`, `vault_log_add`, `vault_events_emit`, `vault_adr`, `vault_project`, `vault_handoff`, `vault_closeout`, `vault_derived_write`, `vault_source_store` and `vault_optimize`. Each maps a closed set of actions to **one fixed script and subcommand** — there is no "run a script" tool. Arguments are typed by a closed schema and checked again: slugs, task ids and dates by pattern (a whole-string match: a trailing newline does not pass), every text field one line — no ASCII control and no Unicode line or paragraph separator (`\x0b \x0c \x1c-\x1e \x85 U+2028 U+2029`), because `gt_adr merge` and the log merge split on all of them and a separator would forge a heading or a second log line; only an ADR body and a source's content may span lines, and the ADR body is checked for a forged `## ADR-N` heading after every such separator — paths through the same resolver as `vault_read` (inside the vault, never a locked folder) with no leading `-`, passed after `--` where the script supports it, so a path is never an option. The vault and the session are the server's own, never arguments. The script is the one the skill's shell step runs (the vault's own `tools/` copy, or gt's installed one), so its checks apply unchanged: content goes through the write queue and the broker (claims, dedupe, escalation). The server writes vault content itself in exactly two places: a **new** `Sources/` file (`vault_source_store`, an exclusive create — never an overwrite) and the body of an ADR slot its own session just allocated. Settings, install and wiring (`gt_settings.py set`, `gt_sandbox.py`, `gt_doctor.py --fix`, `vault_init.py fresh` / `install-core-rules`), whole-vault rewrites (`gt_upgrade.py run`, a project rename, merge or move), `gt_sync.py` and Core-rule files are **not** offered: the skills say to run them from a terminal. Two reports write gt's own state outside the vault (`catchup`: `~/.claude/golden-thread/state/last-open.json`; `handoffs_surface`: `surface/seen.json`), so `vault_report` is not marked read-only. `vault_derived_write`'s two review-queue actions are not blind overwrites: `gt_lint.py --queue` replaces `review-queue.md` itself (atomically, keeping ticks, copying aside anything it did not generate), run under the broker's drain lock (POSIX only: on Windows the lock is not held, as for the broker itself) and only when no other live session claims the file; gt-wiki's `wiki_lint.py --queue` would clobber it with a plain open, so it runs against a scratch file and its output is queued as a `replace-file` — into `review-queue.md` only when that file is absent, a new vault's empty stub or wiki-lint's own untouched output, else into `review-queue-wiki.md`. Under sandbox mode a session is not registered (`gt_session.py` is skipped), so it records no claim of its own; every server write still carries the server's session and is held by another live session's claim. With unlock on, a report needs `gt:vault:read`; every write tool (and `vault_queue_write` / `vault_queue_drain`) needs `gt:vault:write`, which is never opened by `read_without_unlock`. The `design.md` / `global-memory/` refusal comes before that gate.
- **Hooks and the broker** run outside the sandbox by Claude Code's design, so gt's enforcement keeps working.

8.3 WHAT IT STOPS, PER PLATFORM

PLATFORM	ENFORCED	AGAINST THE ASSISTANT'S SHELL
macOS	Seatbelt (OS) + permission rules	**A boundary** for the shell and every process it starts: writing the vault or gt's state, reading the unlock home, LOTR or the vault is refused by the kernel — as long as no settings file loosens it (8.1) and gt's own channels hold (the inbox broker reads only what the writer could; see 8.2)
Linux	bubblewrap (OS) + permission rules; needs `bubblewrap` and `socat` that **run** — gt executes `bwrap --ro-bind / / true` and `socat -V` (0.20.1; on Ubuntu 24.04 AppArmor makes an unprivileged bwrap fail with "setting up uid map: Permission denied" although it is on PATH). gt refuses to turn the mode on when either fails, before gt unlock asks for a code, with the fix (the package command for your distribution, or the AppArmor setting), because with `failIfUnavailable` Claude Code would refuse to start; `gt_sandbox.py apply --force` writes the rest but never `failIfUnavailable`	Same as macOS
WSL2	bubblewrap, as Linux	Same as macOS
Native Windows	**Permission rules only** — Claude Code has no sandbox there	**Friction, not a boundary**: the file tools are refused, but a script the assistant runs can still open any file. `/gt:gt-doctor` and `gt_unlock.py verify` say "friction"

Verified on macOS (2026-10-03) with the sandbox runtime Claude Code builds on (`@anthropic-ai/sandbox-runtime` 0.0.78), using the lists gt generates and the working directory inside the vault: a write to the vault and a read of a vault page were both refused with `Operation not permitted` , a write to `~/.gt-inbox` succeeded, a hard link to a vault file and a write through a symlink into the vault were refused, and `gt_write_queue.py` run inside it queued to the inbox and was applied by a drain outside. Claude Code's translation of `settings.json` into that runtime, and the file-tool permission rules, were checked by configuration (`claude sandbox status` reports the sandbox enabled and strict), not by a live Claude session. On 2026-10-04 the same runtime ran the review's inbox race for real in a live test: a writer inside it swapped an inbox file for a symlink to the TOTP seed about 8,000 times while the broker drained about 11,000 times outside — nothing reached the vault, where the pre-fix broker copied the seed in in the same test; the runtime itself refused a hard link to the seed.

Verified live on macOS (2026-10-04, Claude Code 2.1.289) for the `.credentials.json` entry: a headless `claude -p` session still answered with it in `denyRead` , and a sandboxed `cat` of a read-denied fixture file got `Operation not permitted` where the same file, not denied, was read. On macOS the real file does not exist (the login is in the Keychain), so the fixture stood in for it; Linux and WSL, where the file holds the login, were not run live. Claude Code reads its login in its own process, which the sandbox does not wrap.

8.4 WHAT IT DOES NOT STOP

- **Malware or any other program running as you.** The fence is around Claude's tools only; your terminal, editors, scheduled jobs and anything else you run are unaffected.
- **Claude Code's own processes outside the sandbox:** hooks, MCP servers (including LOTR and gt-vault, which can read the vault by design, and every other MCP server — section 9), plugin monitors, the status line. Commands you type at the `!` prompt usually run unsandboxed too.

- **What the vault MCP returns.** The assistant can still read the vault through it — that is the point; locked folders stay out of reach.
- **Network exfiltration of what the assistant can read.** gt pre-allows no domain; Claude Code's proxy asks before a sandboxed command reaches a new host (see the sandboxing page on domain fronting).
- **Native Windows** beyond the file tools (above).
- **A repository's own settings loosening the sandbox** (8.1) — unless you make it admin-required with managed settings or `claude --settings` (`gt_sandbox.py managed`). `gt_sandbox.py check` reports it; it does not prevent it.

8.5 WHAT CHANGES FOR YOU

- Restart Claude Code after switching. Turning it **off** has to be done from a terminal: the sandbox write-protects `~/.claude` from the assistant's shell, by design.
- The setting is the one way on. `gt_sandbox.py apply` run by itself while `sandbox_mode` is off is refused with the command to use (0.20.1): it used to write the deny rules while the setting — and with it the gt-vault MCP (`vault_mcp auto`) — stayed off, so after a restart the vault was unreachable, and it skipped gt unlock's confirmation. `gt_sandbox.py remove` always works.
- **Rolling back** to a gt older than 0.20.1 (which has no `gt_sandbox.py` and no `gt_unlock.py`) is refused by `install.sh` while sandbox mode or gt unlock is on, or while gt's sandbox entries are still in `settings.json`, with the exact commands to switch them off first (0.20.1): the older release could never remove them. A rollback to a release that carries `gt_sandbox.py` is allowed.
- gt's vault tools run from the assistant's shell (`gt_tasks`, `gt_lint`, `gt_log`, ...) cannot read or write the vault. Every skill says which gt-vault tool takes each such step instead (0.20.2); the few that need you — settings, install, upgrade, sync, Core rules — it hands you as one command for a terminal. `sandbox_vault_reads allow` gives the shell read access back (writes stay denied).
- `design.md` and `global-memory/` are yours: no gt MCP tool edits them, so under sandbox mode too the assistant shows you the text and you apply it in an editor. A `#conflict` task (a write the broker held for you, or two competing writes) points at a conflict file with every version; decide, edit, and close the task.
- Test receipts, worker declarations and other state under `~/.claude/golden-thread` cannot be written from the shell: run a release's test suite from a terminal.
- Git over SSH and `open` / `osascript` fail inside Claude Code's sandbox (its troubleshooting section has the workarounds); network hosts are asked for one at a time.
- Locking or unlocking a vault folder with `gt_lock.py` updates the locked-folder rules at once.

Check it any time: `gt_sandbox.py status`, `gt_sandbox.py check` (drift), `gt_unlock.py verify` (rows `sandbox-*`; run it from inside Claude Code and the `sandbox-live` row tries a real write to the vault and expects the OS to refuse it), `/gt:gt-doctor` (row `sandbox`).

9. MCP servers outside LOTR

gt unlock, LOTR's tiers and consent, and gt sandbox mode cover **only** what passes through LOTR (`gt-lotr`) and the vault server (`gt-vault`). Every MCP server Claude Code connects to **directly** is **outside both features** — not a weaker form of them, outside them:

- **Not gated by unlock.** Claude Code starts and calls the server itself; no gt scope is asked for, so no factor, tier, consent prompt, unattended rule or audit entry applies.
- **Not contained by sandbox mode.** Claude Code runs MCP servers outside its sandbox by design ("Claude's file tools, MCP servers, and hooks run outside it"), so a direct server reads and writes whatever you can, and reaches any host.
- **Its credentials are not sealed.** Headers and env values in `~/.claude.json` or `.mcp.json` are usually plaintext; Claude Code's own OAuth logins for remote servers are held by Claude Code, where gt cannot seal them.

That covers user and local servers in `~/.claude.json`, a project's `.mcp.json`, servers shipped by any plugin other than gt's, the claude.ai connectors your account has connected, the built-in Claude in Chrome (it drives your logged-in browser profile), and servers in `managed-mcp.json`.

See them. `gt_mcp_inventory.py` (in `~/.claude/golden-thread/hooks/`; `--json` for scripts) lists every server Claude Code may load from the current folder: name, source, transport, state, GATED or UNGATED, and the *kind* of place its credentials live (headers in config, env in config, a headers helper, Claude Code's OAuth store, server-side, browser profile, none). It never

prints a header, env, argument or URL value. `/gt:gt-doctor` row `mcp` lists the ungated ones, and `gt_unlock.py verify` row `mcp` is **NOT-CHECKED — never PASS — while any exist**, and the level line says the L1/L2 claim covers LOTR and gt-vault only. GATED means named `gt-vault` or `gt-lotr` **and** running gt's own script (`gt_vault_mcp.py` , `lotr_mcp.py`); a server merely named like one is UNGATED.

Route through LOTR. A remote HTTP server can be put behind LOTR today as an `mcp` connection (`lotr add-mcp …` , see the gt-lotr skill), then removed from Claude Code with `claude mcp remove <name>` , which also deletes Claude Code's copy of its login. It is then gated like any LOTR connection. stdio servers are planned (0.21) and stay outside until then. A remote server that wants an OAuth login can have LOTR hold it (section 9.1) instead of reusing Claude Code's. To stop a server without moving it, disable it in `/mcp` , or remove it.

Enterprise: make it a boundary. Only managed settings can stop a user or repository adding servers. gt **prints** this recipe and never writes it (`gt_mcp_inventory.py managed`):

```
{
  "allowManagedMcpServersOnly": true,
  "allowedMcpServers": [{"serverName": "gt-vault"}, {"serverName": "gt-lotr"}],
  "deniedMcpServers": [{"serverName": "claude-in-chrome"}]
}
```

Merge it into `managed-settings.json` (or a file in `managed-settings.d/`) at `/Library/Application Support/ClaudeCode/` (macOS), `/etc/claude-code/` (Linux, WSL) or `C:\Program Files\ClaudeCode\` (Windows); servers you provide centrally go in `managed-mcp.json` in the same folder. `serverName` is a label, not a security control — prefer `serverCommand` / `serverUrl` entries where you can — and a `deniedMcpServers` match always wins. Check the result with `claude mcp list` .

9.1 LOTR AS THE OAUTH CLIENT (`LOTR CONNECT` , GT-LOTR 0.3.0)

`lotr connect NAME --url https://…/mcp` signs in to a remote MCP server at your terminal (MCP authorization spec 2026-07-28: PKCE S256, `resource` , loopback redirect) and keeps the login inside LOTR, so the server is gated like any LOTR connection. What that does and does not give:

Using gt-lotr adds no new attack surface for a process running as you. Each downstream service is reachable with the same credentials whether or not you use the gateway. The gateway adds a local socket and a token store, both readable only by your user. It never gives the model a token, and its controls limit what the model does on its own. It doesn't stop a process already running as you.

Scope of protection. gt is built to stop the mistakes and ordinary misuse that a person or an assistant working in a closed, local setup would make. It does not stop an attacker who already runs code as you on this machine. Anything running as your user can act as you, and gt doesn't block that. If the machine is compromised, gt can't protect it. We keep hardening gt against deliberate attempts, and we test those attempts to learn what is possible. Until the hardening is complete, treat gt's protections as protections against accidents and against misuse through the tool, not against a hostile process with your access.

ITEM	WHERE IT LIVES	RATING
Refresh token, gt unlock on	`sealed:` in the authority (re-sealed on rotation without a prompt on macOS; Windows asks Hello for the re-seal: a daemon with no person present then keeps the rotated token in memory only and reports it)	**L2**
Refresh token, unlock off or `--l1`	a mode-600 file the store wrote	**L1**: any process of your user can read it
Access token	lotrd's memory only, per calling seat, never written	gone at daemon exit
Client id / secret	registry ref (`sealed:` / `store:`); a secret never appears in the registry, argv or output	as the refresh token

- The access token **outlives the unseal that minted it** for at most the server's `expires_in` (capped by `max_access_cache_s`, default 3600 s). While gt unlock demands a fresh factor for every unseal, one unseal therefore serves that many minutes of calls for that seat. Lower `max_access_cache_s` in the connection's `auth.oauth` block to tighten it.
- An OAuth connection **never** serves an unattended write or consent (a hub client gets `oauth_unattended_refused`). **Unattended reads are supported only with an L1 `store:` refresh token** (connect with `--l1`; any process of your user can read that file, which is the honest cost). A hub client asking for a read on a connection whose refresh token is `sealed:` is refused with `oauth_unattended_refused` before any request is made -- adding a `secret:<ref>` allow-list entry does NOT make it work, because the seat that would use the opened token could not re-store the rotation, and presenting an already-rotated token ends the sign-in at the server.
- `lotr connect` asks the server's authorization server for its metadata, so the issuer and every endpoint it names are fetched by LOTR: https only, redirects only within an origin, private / loopback / link-local / cloud-metadata addresses refused (the metadata addresses even with `--allow-private-network`), every response size- and time-limited.
- Tool tiers are unchanged and still derived from the server's own annotations and names, which are hints a server controls (hardening is the open 0.21 item): a server that mislabels a destructive tool as read-only can have it run at the read tier.
- It does not sign Claude Code out of its own copy of a login; `claude mcp remove <name>` does.
- **Reuse detection can end a sign-in.** Rotating refresh tokens are one-time: a server that follows RFC 9700 revokes the whole grant when a used one is presented again. LOTR therefore never presents a token it knows is stale, and a caller that cannot re-store a rotated token (an unattended hub client with a sealed refresh token) is refused before any request is made. If a grant ends anyway, run `lotr login <name>`: it signs in again and revokes the old token.
- **A sign-in is started by you.** `lotr connect|login|disconnect` refuse to run from inside Claude Code's process tree and, with gt unlock on, ask for the hub-admin step-up. A program running as you can still detach from that tree: the guard stops a model's Bash tool, not malware. Without gt core the process tree cannot be read; the `CLAUDECODE` marker is then a convenience only (a caller can unset it), and a sign-in with no terminal on stdin is refused.
- **Direct connections only.** LOTR ignores `HTTP_PROXY` / `NO_PROXY`, so a token never passes through an environment or system proxy (a proxy-only network cannot use OAuth connections). TLS verification trusts the platform store, so `SSL_CERT_FILE` / `SSL_CERT_DIR` in lotrd's environment are part of the trust boundary. On native Windows a `store:` file is protected by the profile directory's ACL, not by mode bits.
- **Tiers on OAuth connections only ever go up from the server's hints**: a name containing a risky word (delete, remove, send, write, create, update, merge, post, drop, grant, revoke, erase, unlink, edit, modify, add_comment, reply, replace, empty, ...; the name is NFKC-folded with format characters removed first) as a substring of the name is consent, read prefix or not (`get_deleteall` is consent); a short allow-list of read words that contain one (settings, assets, presets, ...) is the only exception, and some reads (`list_commits`) are consent by that rule until the owner pins them with `policy.read`.
- **A sealed secret is bound to its record.** `registry.json` can be edited by any process of your user, and lotrd posts the refresh token to the token endpoint the record names. A `sealed:` refresh token or client secret is therefore stored inside an envelope whose SHA-256 digest covers the record's `issuer`, `token_endpoint`, `revocation_endpoint`, `client_id` and `resource`. If any of those no longer matches, or a sealed ref holds a value with no envelope, lotrd and the CLI refuse with `oauth_binding_mismatch` and send nothing. An L1 `store:` OAuth secret is wrapped the same way, so a copy of it pointed at by a bearer connection is refused (the bearer paths refuse every envelope). What this does **not** cover: an L1 file is still readable by any process of your user, and a value written before this change is plain until its next refresh (`lotr status` says which connections are still plain), and the fields outside the digest (the authorization endpoint, scopes) are not bound. **A sealed OAuth connection made before the binding existed is refused the same way: run `lotr disconnect <name>`, then `lotr connect <name> --url ...` once.** A record edited away from `scheme: oauth` (to `bearer`, keeping the OAuth secret's ref) is refused by the registry, and the bearer paths refuse to send an envelope at all, so the downgrade cannot turn the sealed token into a bearer header.
- **The record is checked again at use.** The connection's `endpoint` must be the `resource` the sign-in was issued for (`oauth_resource_mismatch`, at registry load and before every token use), and an OAuth `token_ref` may only be a `sealed:` or `store:` ref, so an edited record cannot point the refresh at another file or environment variable.
- **The unattended refusal comes before any dialog.** A hub client's write or consent request to an OAuth connection is refused (`oauth_unattended_refused`) before a consent dialog is raised: nobody is asked to approve something that cannot happen.
- **Scope hints need no quoting.** `--scope` may be repeated, and a hint built from a server's scope list is printed as `--scope a --scope b`; a scope word that is not plain letters, digits and `: . _ / + = @ -` (starting with a letter or digit) is left out of the hint altogether.
- **Translated IPv4 forms.** `::ffff:0:a.b.c.d` (RFC 2765) is judged as the IPv4 address it carries, like `::ffff:a.b.c.d` and NAT64; a DNS lookup counts against the same total time budget as the connect.

- **Local names as the authorization endpoint** (design): `localhost` and `*.localhost` are refused there, so the browser is never sent to this machine by name.
- **The browser.** `GT_LOTR_NO_BROWSER=1` makes `connect` / `login` print the sign-in URL and never open a browser, `--open-browser` included. Without it a browser is opened only when stdin is a terminal, or when you pass `--open-browser`.
- **Pre-registered clients (`--client-id`): what is tested and what is not.** The flags are `--client-id`, a secret from `--client-secret-file`, `--client-secret-stdin` or `--client-secret-prompt` (never argv; stored as a ref like the refresh token), `--no-resource` (omit the RFC 8707 `resource` parameter for a provider that rejects it; the token is then not audience-bound by that parameter) and `--redirect-host localhost|127.0.0.1` (the name in the redirect URI; the listener binds 127.0.0.1 either way). **The Google and Microsoft paths are tested only against a fake authorization server** (`dev/fake_oauth_mcp.py`); no sign-in against either real provider has been run. Linear (dynamic registration, no flags) is the provider the live check is written for: `dev/oauth-live-check-linear.md`.

10. gt 0.21.0: installed-code integrity, action tokens, I/O brokers, the testing profile

Four additions, each a setting with a plain reading of what it protects and what it costs. None changes behaviour when it is off. None is a defence against something already running as you that can wait for you to approve.

10.1 THE `PARANOID` GROUP: PROTECTING GT'S OWN CODE

`paranoid` (off by default) answers "can something quietly rewrite the programs that keep the assistant in check?" It acts on gt's installed **code** only (hooks, packs, the plugin cache and the marketplace); working data is never locked. Each part follows the master unless set, and is a security setting:

SETTING	WHAT IT DOES
`paranoid.os_lock`	`uchg` (macOS), read-only plus an ACE (Windows) or `a-w` (Linux) on the installed code; bytecode is compiled first
`paranoid.code_write_gate`	the presence gate `gt-code-write` (scope `gt:code:write`): the installer, component self-repair and uninstall unlock through it
`paranoid.bash_guard`	a PreToolUse(Bash) guard that asks or denies (never allows) writes and flag changes to protected paths
`paranoid.signed_manifest`	each hook verifies a signed manifest of its own files before it runs (about 0.45 ms) and fails closed if they were tampered with; signed by the unlock key, else pinned and labelled weaker
`paranoid.plugin_dirs`, `paranoid.require_unlock`	the plugin cache and marketplace under the same protection; refuse to turn paranoid on until unlock is set up

`gt_integrity.py status`, `gt_settings.py show` and `gt_doctor` read `paranoid (partial)` when a part is off, and the doctor has an `integrity` row. **Limits:** an owner who unlocks can change anything; a process that runs as you while the code is unlocked can too.

10.2 SINGLE-USE ACTION TOKENS

`paranoid.action_tokens` makes each listed action need a token you mint for that one action:

`disk_read`, `disk_list`, `net_fetch`, `disk_write`, `lotr_read`, `lotr_write`, `lotr_consent`, `vault_read`, `vault_write`, `secrets_unseal`, `git_push`, `git_commit`, `publish`, `grant_add`, `plan_bound` (each is `paranoid.action_tokens.<action>`, default `follow`, so the master decides until you set one).

```
python3 ~/.claude/golden-thread/hooks/gt_action_token.py mint <action> <target>      # in a terminal; as
python3 ~/.claude/golden-thread/hooks/gt_action_token.py mint-plan <plan.json>      # an approved plan,
```

A token is HMAC-signed, bound to the action, the **exact target** and the content, expires in minutes and is **spent once** by an atomic rename. Replay, retarget, expiry and forgery are refused, and the refusal names the mint command as the route. The assistant is given only the token's id. Hooks that run a command read the token from the `GT_ACTION_TOKEN` environment variable, which is never stored. The audit chain is keyed and holds no values (`gt_action_token.py verify`). **Exemption:** gt-lotr's own daemon unsealing its own credentials is exempt from `secrets_unseal` , because it is gt acting for the gateway rather than the assistant asking. Doctor skips the write-queue smoke checks while the `vault_write` token is required.

10.3 I/O BROKERS: THE ASSISTANT'S DISK AND NETWORK, ONE CAPABILITY EACH

TOOL (GT-VAULT MCP)	AGENT	RULE
`disk_read` , `disk_list`	`disk-reader`	free inside the repositories a project's `source.md` names; elsewhere presence, or a live timed window; real path resolved first; credential-shaped text withheld
`net_fetch`	`net-fetcher`	GET only, no cookies or credentials; `io_fetch_allow` domains served, other public domains ask presence; private, loopback, link-local and metadata addresses never; every DNS answer checked, connection pinned, redirects re-checked
`disk_write`	`disk-writer`	one text file inside a git work tree; presence on **every** write or a timed window for one repo; backup first; never executable; no commit, push or subprocess

`disk_write` never writes `~/.claude` , shell start-up files, LaunchAgents/Daemons, `~/.ssh` , any `.git` component, `.env` / `*.env` or key files, `.envrc` , `.gitattributes` , `.gitmodules` , credential dotfiles or the vault, and refuses CI workflow files unless the owner sets `io_write_ci=allow` . It also refuses when no presence method is enrolled and Claude Code is set to run the tool without asking. An independent review of the write path produced seven fixes (a work-tree root of `/` or home, executable overwrite, `*.env` , case-insensitive matching, Windows alternate data streams and UNC roots, a prompt cool-down); two Windows/POSIX race windows remain documented. The gates `io.read` , `io.fetch` and `io.write` are always on; `io_write_presence` cannot be turned off.

10.4 THE TESTING PROFILE

`gt_settings.py testing on` pre-approves test runners, static scanners and gt's own dev test commands for one project so an unattended run stops asking. It records exactly what it added, backs up first, and `off` removes only that. It never writes modes, hooks, skip-permissions or `GT_TEST_GATE` , so every Core-rule gate (the vault-write guard, the secrets Stop hook) is untouched.

10.5 HOOKS AND THE INSTALL

Since 0.21.0 gt's hooks ship in each plugin's `hooks/hooks.json` and dispatch through `~/.claude/golden-thread/wired/` (Claude Code 2.1.136 and newer; older versions keep `settings.json` entries pointing into `~/.claude/golden-thread/hooks/`). A new install **asks the security questions first** (`install.sh --questions`); not at a terminal it exits 5 rather than take a security default silently, and `--accept-defaults` is the explicit opt-out.